

VLSI INTERNSHIP

WEEK 3 REPORT

Task 3: Verilog RTL Design of Sequential Circuits and Flip-Flops

Name	Neel Vyas
College	G H Patel College of Engineering and Technology
Department	Electronics and Communication Engineering (ECE)
Intern ID	MT4900
Duration	6 Weeks (14 May 2026 – 29 June 2026)
Submitted To	Maincrafts Technology
Date	June 2026

1. Introduction

This report documents the work completed for Task 3 of the VLSI Design Internship at Maincrafts Technology. Task 3 is about Verilog RTL Design of Sequential Circuits and Flip-Flops — moving from the combinational circuits of Task 2 into time-driven, state-retaining designs that make up the core of every real processor and controller.

In Task 2, I built and verified Logic Gates, a Half Adder, and a Full Adder using continuous assign statements. Task 3 builds on that Verilog foundation by introducing the `always @(posedge CLK)` block, non-blocking assignment (`<=`), and the concept of stored state. The four circuits implemented here are a D Flip-Flop, a JK Flip-Flop, a 4-bit Register, and a 4-bit Binary Counter — all simulated in Icarus Verilog and verified in GTKWave.

2. Concepts Studied

Concept	Explanation
Sequential Logic	Output depends on current inputs AND previous state — requires memory elements
Flip-Flop	Basic 1-bit storage element; captures data on a clock edge and holds it until the next
Clock Signal	Periodic square wave that synchronises all state transitions in the circuit
<code>always @(posedge CLK)</code>	Verilog construct that triggers the block only on the rising edge of the clock
Non-blocking Assignment (<code><=</code>)	Schedules updates end-of-timestep — essential for correct sequential RTL
Register	Group of D flip-flops sharing a clock; stores a multi-bit data word
Counter	A register that auto-increments its value on every clock cycle

2.1 Sequential vs Combinational — Key Difference

Combinational circuits (Task 2) produce outputs purely from current inputs — change the inputs and the outputs follow immediately. Sequential circuits remember their previous state: output depends on both current inputs and what happened before.

In Verilog this maps directly to syntax: combinational logic uses `assign` or `always (*)`, while sequential logic uses `always @(posedge CLK)` with non-blocking assignments. The `<=` operator is critical — it ensures all flip-flops sample their inputs simultaneously before updating outputs, which matches how real silicon behaves.

3. Tools and Simulation Workflow

Tool	Purpose	Type
Icarus Verilog (iverilog)	Compiles Verilog source into a simulation executable	Free / Open Source
vvp	Runs the compiled simulation and produces the .vcd file	Bundled with Icarus
GTKWave	Waveform viewer — loads .vcd and displays signal traces	Free / Open Source

3.1 Simulation Steps (same for all four circuits)

```
# Step 1 — Compile design + testbench
iverilog d_flipflop.v tb_d_flipflop.v

# Step 2 — Run simulation (generates .vcd file)
vvp a.out

# Step 3 — Open waveform in GTKWave
gtkwave d_flipflop.vcd
```

4. Part A — D Flip-Flop

4.1 Design Logic

The D Flip-Flop is the simplest and most widely used sequential element. It has one data input D, a clock CLK, and output Q. On every rising clock edge, Q captures whatever D holds at that moment and locks it in until the next rising edge. Between clock edges, Q is completely frozen regardless of D activity.

4.2 Truth Table

Clock Edge	D (Input)	Q (Next State)
Rising Edge (↑)	0	0
Rising Edge (↑)	1	1
No rising edge	X (any)	Q (unchanged)

4.3 Verilog Module Code (d_flipflop.v)

```
module d_flipflop (
    input D,
    input CLK,
    output reg Q
);
always @(posedge CLK)
begin
    Q <= D;
end
endmodule
```

4.4 Testbench Code (tb_d_flipflop.v)

```
module tb_d_flipflop;
    reg D;
    reg CLK;
    wire Q;
    d_flipflop uut (
        .D(D),
        .CLK(CLK),
        .Q(Q)
    );
    initial begin
        $dumpfile("d_flipflop.vcd");
        $dumpvars(0, tb_d_flipflop);
    end
    // Clock generation
    initial begin
        CLK = 0;
        forever #5 CLK = ~CLK;
    end
    // Stimulus
    initial begin
        D = 0; #10;
        D = 1; #10;
        D = 0; #10;
        D = 1; #10;
        D = 0; #10;
        $finish;
    end
end
endmodule
```

4.5 GTKWave Waveform — Simulation Screenshot

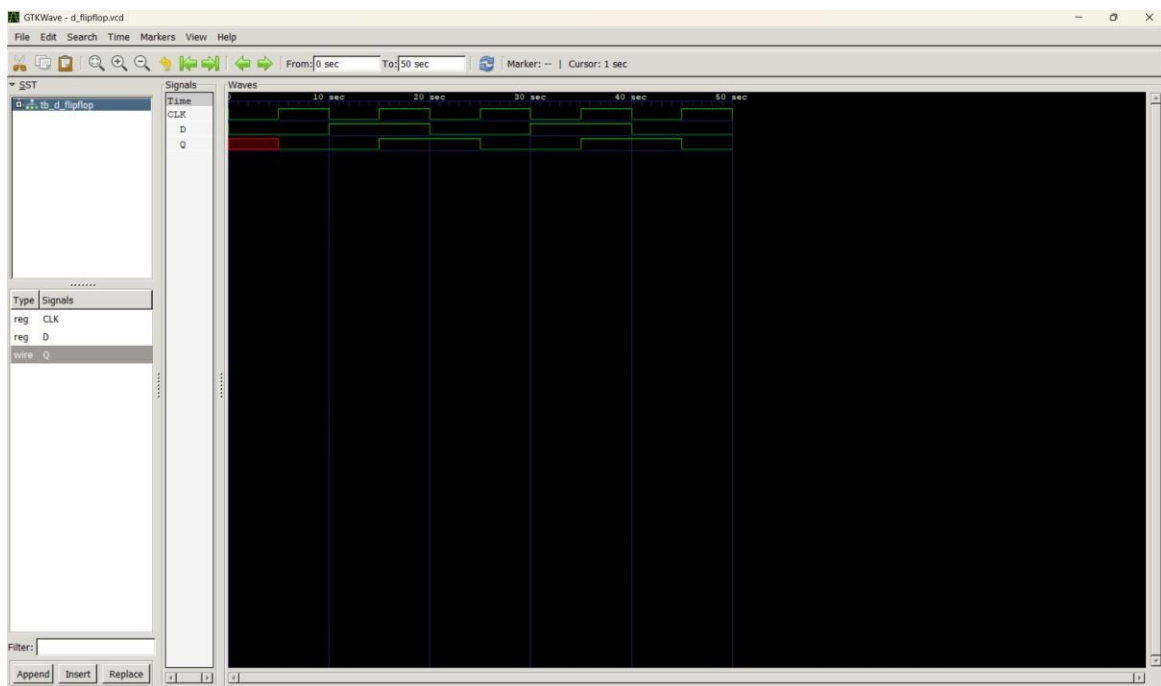


Figure 1 — GTKWave: D Flip-Flop simulation (d_flipflop.vcd) — CLK, D, Q

4.6 Waveform Analysis

- Q initialises to unknown (red) before the first clock edge — expected for an uninitialised flip-flop.
- On every rising CLK edge, Q captures the current value of D and holds it steady until the next edge.
- D changing between clock edges has zero effect on Q — it remains frozen until the next posedge.
- All transitions match the truth table exactly: Q follows D with one clock cycle of latency.

Key observation: The one-cycle latency between D and Q is the defining feature of edge-triggered storage — this is precisely what separates a flip-flop from a transparent latch.

5. Part B — JK Flip-Flop

5.1 Design Logic

The JK Flip-Flop extends the SR flip-flop by resolving its forbidden state. Two inputs J (Set) and K (Reset) produce four distinct, well-defined behaviours on each rising clock edge: Hold, Reset, Set, and Toggle. The Toggle state — where Q inverts itself every clock cycle when J=K=1 — is what makes the JK flip-flop uniquely powerful for building frequency dividers and ripple counters.

5.2 Truth Table

J	K	Q (Next)	Operation
0	0	Q	Hold — output retains its current value
0	1	0	Reset — output cleared to 0
1	0	1	Set — output driven to 1
1	1	$\sim Q$	Toggle — output inverts on every clock edge

5.3 Verilog Module Code (jk_flipflop.v)

```
module jk_flipflop (
    input J,
    input K,
    input CLK,
    output reg Q
);
always @(posedge CLK)
begin
    case ({J,K})
        2'b00: Q <= Q;      // No Change
        2'b01: Q <= 1'b0;   // Reset
        2'b10: Q <= 1'b1;   // Set
        2'b11: Q <= ~Q;     // Toggle
    endcase
end
endmodule
```

5.4 Testbench Code (tb_jk_flipflop.v)

```
module tb_jk_flipflop;
    reg J, K, CLK;
    wire Q;
    jk_flipflop uut (
        .J(J),
        .K(K),
        .CLK(CLK),
        .Q(Q)
    );
    initial begin
        $dumpfile("jk_flipflop.vcd");
        $dumpvars(0, tb_jk_flipflop);
    end
    // Clock generation (10 ns period)
    initial begin
```

```

    CLK = 0;
    forever #5 CLK = ~CLK;
end
// Test cases
initial begin
    J = 0; K = 0;    // No Change
    #10;
    J = 0; K = 1;    // Reset
    #10;
    J = 1; K = 0;    // Set
    #10;
    J = 1; K = 1;    // Toggle
    #10;
    J = 1; K = 1;    // Toggle again
    #10;
    J = 0; K = 0;    // No Change
    #10;
    $finish;
end
endmodule

```

5.5 GTKWave Waveform — Simulation Screenshot

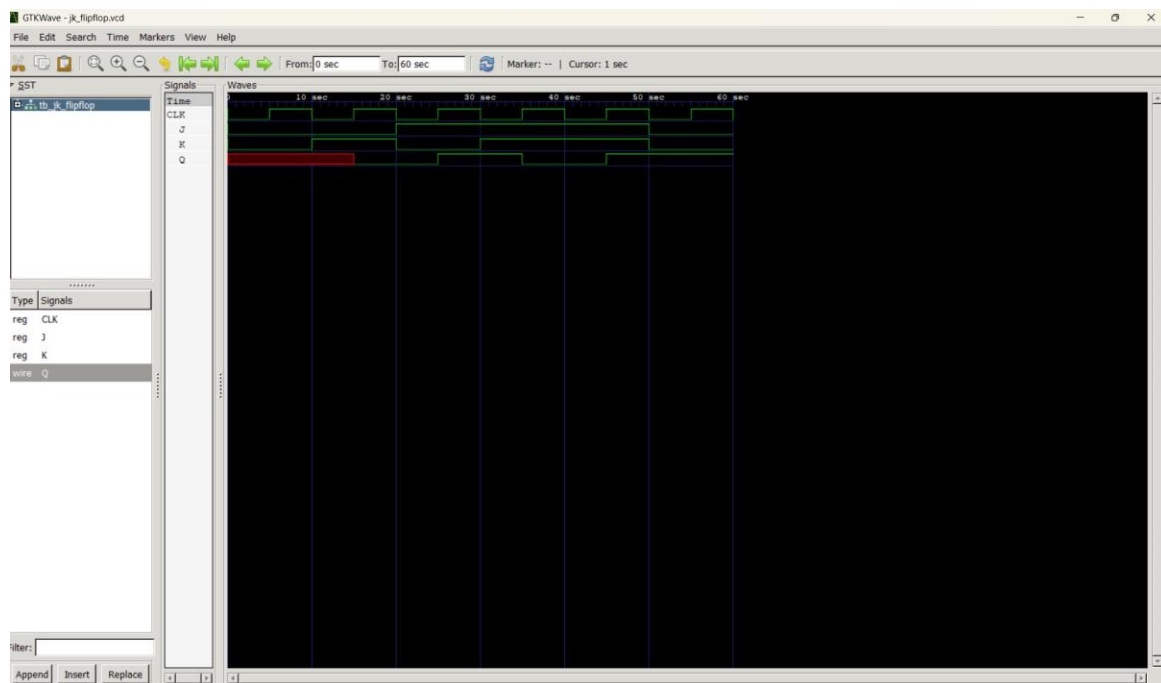


Figure 2 — GTKWave: JK Flip-Flop simulation (jk_flipflop.vcd) — CLK, J, K, Q

5.6 Waveform Analysis

- Hold ($J=0, K=0$): Q stays flat across clock edges — no state change occurs.
- Reset ($J=0, K=1$): Q drops to 0 on the next rising clock edge regardless of its previous state.
- Set ($J=1, K=0$): Q rises to 1 on the next rising edge.
- Toggle ($J=1, K=1$): Q alternates between 0 and 1 on every rising clock edge — the output frequency is exactly half of CLK, confirming divide-by-2 behaviour.

Key observation: The toggle mode is unique to the JK flip-flop. No other basic flip-flop type can produce this behaviour from a single clock input, which is why JK flip-flops are used as the building blocks of asynchronous (ripple) counters.

6. Part C — 4-bit Register

6.1 Design Logic

A register is a group of D flip-flops sharing one clock, designed to store a multi-bit word. A 4-bit register stores values from 0000 to 1111 and loads a new 4-bit value on every rising clock edge. In Verilog this is written identically to a 1-bit D flip-flop — only the [3:0] port widths change. The synthesiser maps it to four parallel D flip-flops, all clocked simultaneously for a synchronous parallel load.

6.2 Verilog Module Code (register4.v)

```
module register4 (
    input CLK,
    input [3:0] D,
    output reg [3:0] Q
);
always @(posedge CLK)
begin
    Q <= D;
end
endmodule
```

6.3 Testbench Code (tb_register4.v)

```
module tb_register4;
    reg CLK;
    reg [3:0] D;
    wire [3:0] Q;
    register4 uut (
        .CLK(CLK),
        .D(D),
        .Q(Q)
    );
    initial begin
        $dumpfile("register4.vcd");
        $dumpvars(0, tb_register4);
    end
    // Clock generation
    initial begin
        CLK = 0;
        forever #5 CLK = ~CLK;
    end
    // Test cases
    initial begin
        D = 4'b0000; #10;
        D = 4'b1010; #10;
        D = 4'b1100; #10;
        D = 4'b1111; #10;
        D = 4'b0011; #10;
        $finish;
    end
end
endmodule
```

6.4 GTKWave Waveform — Simulation Screenshot

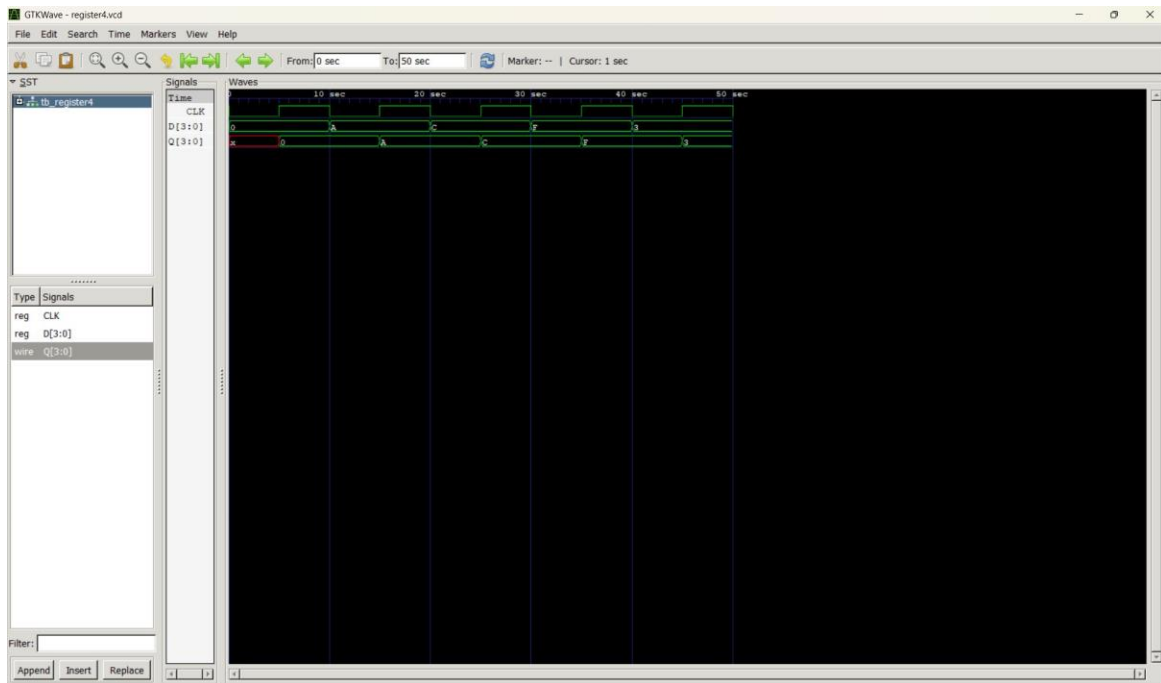


Figure 3 — GTKWave: 4-bit Register simulation (register4.vcd) — CLK, D[3:0], Q[3:0] shown in hex

6.5 Waveform Analysis

- Q starts as unknown (X) before the first clock edge, then loads 0x0 on the first rising CLK.
- Each subsequent rising edge: Q captures the 4-bit hex value of D at that instant — 0, A, C, F, then 3.
- Q is completely stable between clock edges regardless of D — the one-cycle lag between D and Q is clearly visible on the bus trace.
- GTKWave's hex bus display makes it easy to read each loaded value directly: 0 → A → C → F → 3.

Key observation: This is the exact same storage structure used between pipeline stages in a processor. The synchronous load ensures all bits of the word transfer atomically on the clock edge, preventing data corruption.

7. Part D — 4-bit Binary Counter

7.1 Design Logic

A binary counter is a register that auto-increments by 1 on every rising clock edge. A 4-bit counter cycles through 0 (0000) to 15 (1111) and wraps back to 0 — 16 states driven purely by the clock with no external data input. Verilog handles the 4-bit overflow naturally: $4'hF + 1 = 4'h0$, so no explicit wrap logic is needed.

7.2 Verilog Module Code (counter4.v)

```
module counter4 (
    input CLK,
    output reg [3:0] Q
);
initial
    Q = 4'b0000;
always @(posedge CLK)
begin
    Q <= Q + 1;
end
endmodule
```

7.3 Testbench Code (tb_counter4.v)

```
module tb_counter4;
    reg CLK;
    wire [3:0] Q;
    counter4 uut (
        .CLK(CLK),
        .Q(Q)
    );
    initial begin
        $dumpfile("counter4.vcd");
        $dumpvars(0, tb_counter4);
    end
    // Clock generation (10 ns period)
    initial begin
        CLK = 0;
        forever #5 CLK = ~CLK;
    end
    // Run simulation
    initial begin
        #160;
        $finish;
    end
end
endmodule
```

7.4 GTKWave Waveform — Simulation Screenshot

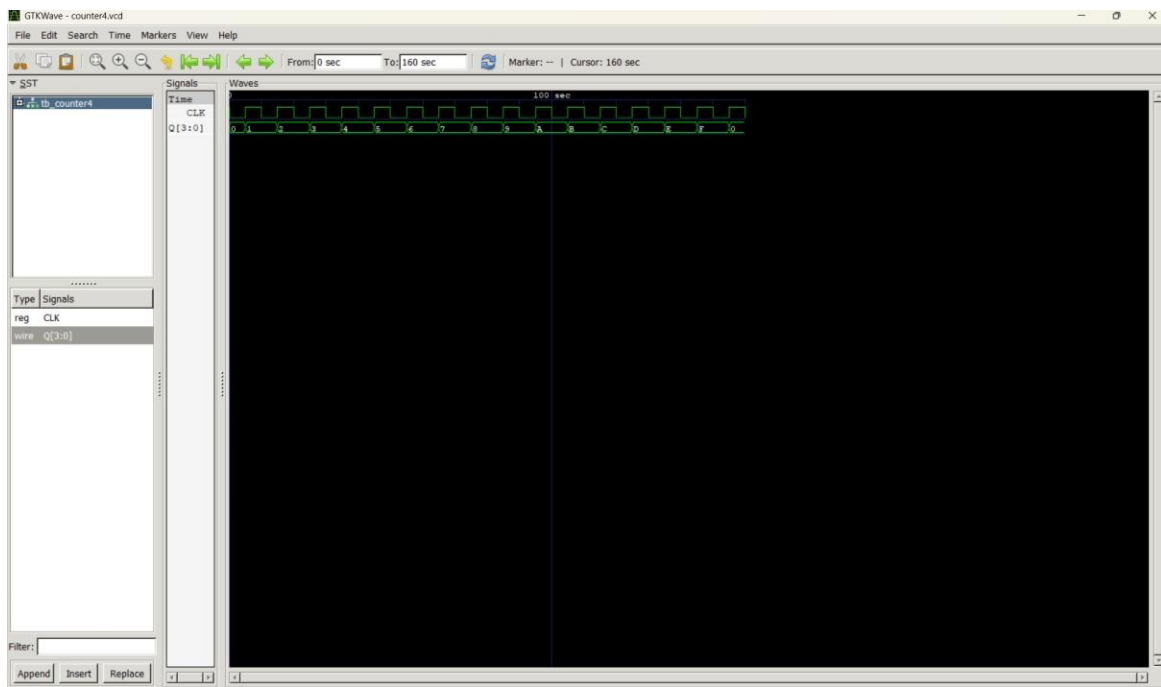


Figure 4 — GTKWave: 4-bit Counter simulation (counter4.vcd) — Q[3:0] counting 0→F→0 over 16 clock cycles

7.5 Waveform Analysis

- Counter starts at 0 (initialised by the initial block) and steps through 1, 2, 3 ... 9, A, B, C, D, E, F on successive rising CLK edges.
- After reaching F (decimal 15), it wraps cleanly to 0 on the next edge — natural 4-bit overflow, no extra logic needed.
- Each hex value occupies exactly one clock period in the waveform, confirming uniform synchronous incrementing.
- The full 16-state sequence (0x0 to 0xF) is verified in 160 time units — 16 clock cycles of 10 units each.

Key observation: The wrap from F back to 0 is the most important verification point. It confirms that Verilog's 4-bit integer arithmetic overflows correctly and that no stuck state or wrong-value transition occurs at the boundary.

8. Key Learning Outcomes

After completing Task 3, I can now:

- Write synthesisable Verilog for sequential circuits using always @(posedge CLK) and non-blocking assignments (<=).
- Implement and verify a D flip-flop (data capture), JK flip-flop (hold/reset/set/toggle), 4-bit register (parallel load), and 4-bit counter (auto-increment with wrap).
- Understand why <= is mandatory in sequential RTL — it models simultaneous flip-flop sampling and prevents race conditions that blocking assignments would cause.
- Generate clocks in testbenches (forever #5 CLK = ~CLK) and run simulations for a set number of clock cycles.
- Read GTKWave waveforms for clocked circuits — matching rising edges to output transitions and verifying multi-bit bus values in hex display mode.
- Recognise the relationship between flip-flops, registers, and counters — all built from the same underlying D flip-flop element, scaled by width and feedback.

9. Deliverables Completed

Deliverable	Status	Location
d_flipflop.v	Completed	Section 4.3
jk_flipflop.v	Completed	Section 5.3
register4.v	Completed	Section 6.2
counter4.v	Completed	Section 7.2
tb_d_flipflop.v	Completed	Section 4.4
tb_jk_flipflop.v	Completed	Section 5.4
tb_register4.v	Completed	Section 6.3
tb_counter4.v	Completed	Section 7.3
Waveform — D Flip-Flop	Completed	Section 4.5 (Figure 1)
Waveform — JK Flip-Flop	Completed	Section 5.5 (Figure 2)
Waveform — 4-bit Register	Completed	Section 6.4 (Figure 3)
Waveform — 4-bit Counter	Completed	Section 7.4 (Figure 4)
Circuit design explanation	Completed	Sections 4.1, 5.1, 6.1, 7.1
Simulation results analysis	Completed	Sections 4.6, 5.6, 6.5, 7.5

10. Conclusion

Task 3 was the most significant conceptual step of this internship so far. Moving from combinational to sequential design is not just a syntax change — it is a completely different way of thinking about

hardware. Instead of asking "what is the output for these inputs?", sequential design asks "how does the circuit evolve over time?"

The D flip-flop taught me the fundamental idea: one clock edge, one state update, everything else frozen. The JK flip-flop showed how previous state can feed back into next state, enabling toggle behaviour that no combinational gate can produce alone. The register showed that scaling from 1 bit to 4 bits is trivial in Verilog. The counter demonstrated how feedback through an adder creates autonomous behaviour driven only by the clock.

GTKWave was essential for all four designs. Waveforms for sequential circuits are far more informative than static truth tables — you have to see the clock edges and the transitions that follow them to really understand what the circuit is doing. Reading those waveforms and verifying them against expected behaviour is a skill I will carry into every future RTL task.

11. References

- Icarus Verilog — Open-source Verilog simulator: <http://iverilog.icarus.com>
- GTKWave — Waveform viewer: <http://gtkwave.sourceforge.net>
- EDA Playground — Online Verilog simulation environment: <https://www.edaplayground.com>
- TutorialsPoint — Verilog HDL Tutorial: https://www.tutorialspoint.com/vlsi_design/vlsi_verilog.htm
- GeeksForGeeks — Digital Logic and Sequential Circuits: <https://www.geeksforgeeks.org/digital-electronics-digital-logic/>